

Jay Mayurkumar Varma

Student Code: iitg_ds_25012083

Handwritten Digit Recognition on MNIST Using Logistic Regression and MLP

Introduction

Handwritten digit recognition is a classic pattern recognition problem with wide applications in postal automation, bank cheque processing, and digitizing handwritten forms. The MNIST dataset is a widely used benchmark for this task, containing grayscale images of handwritten digits (0–9) that allow easy comparison of different machine learning models.[1][2]

In this project, I implement and compare two models on the MNIST dataset:

1. A **multinomial Logistic Regression** model as a baseline linear classifier.
2. A **Multilayer Perceptron (MLP)** deep neural network implemented with TensorFlow/Keras.

The goal is to evaluate how a deeper non-linear model improves performance over the linear baseline, and to analyze errors using a confusion matrix and learning curves.

Dataset and Preprocessing

MNIST Dataset

The MNIST (Modified National Institute of Standards and Technology) dataset consists of:

- 70,000 grayscale images of handwritten digits (0–9)
- Each image is of size 28×28 pixels
- Standard split: 60,000 training images and 10,000 test images[1][2]

In the code, MNIST is loaded directly from `tensorflow.keras.datasets`:

```
(x_train, y_train), (x_test, y_test) = datasets.mnist.load_data()
```

Preprocessing Steps

Each input image is preprocessed as follows:

1. **Flattening:**
 - Original shape: (28, 28)
 - Flattened to a 1D vector of length 784:
`x_train_flat = x_train.reshape(-1, 784)`
`x_test_flat = x_test.reshape(-1, 784)`

2. Normalization:

- Pixel values in MNIST are in the range [0, 255].
- They are converted to float32 and scaled to [0, 1] by dividing by 255:
x_train_flat = x_train_flat.astype('float32') / 255.0
x_test_flat = x_test_flat.astype('float32') / 255.0

These steps ensure that both models receive standardized numeric inputs and that optimization is more stable.

Baseline Model: Logistic Regression

Model Description

The baseline model is a **multinomial Logistic Regression** classifier trained on the flattened and normalized feature vectors. It is implemented using `sklearn.linear_model.LogisticRegression` with the following settings:

```
baseline = LogisticRegression(  
    max_iter=100,  
    multi_class='multinomial',  
    solver='lbfgs',  
    n_jobs=-1  
)  
baseline.fit(x_train_flat, y_train)  
baseline_acc = accuracy_score(y_test, baseline.predict(x_test_flat))
```

Key choices:

- `multi_class='multinomial'`: True multi-class logistic regression rather than one-vs-rest.
- `solver='lbfgs'`: A quasi-Newton method suited for multinomial problems.
- `max_iter=100`: Maximum number of iterations for convergence.
- `n_jobs=-1`: Uses all available CPU cores for training.

Baseline Performance

On MNIST, a well-configured multinomial Logistic Regression model typically achieves around **85–94%** accuracy on the test set, depending on regularization and exact preprocessing settings.[3][4] In this project, the baseline test accuracy is printed as:

baseline accuracy: XX.XX%

(Here, XX.XX is the actual percentage obtained when running the code.)

This baseline serves as a reference to quantify the improvement gained by introducing a deeper neural network.

Deep Model: Multilayer Perceptron (MLP)

Architecture

The main model is a fully connected **Multilayer Perceptron** implemented using TensorFlow/Keras:

```
model = models.Sequential([
layers.Dense(512, activation='relu', input_shape=(784,)),
layers.BatchNormalization(),
layers.Dropout(0.2),
layers.Dense(256, activation='relu'),
layers.Dense(128, activation='relu'),
layers.Dense(10, activation='softmax')
])
```

Architecture details:

- **Input layer:** 784-dimensional vector (flattened 28×28 image).
- **Hidden Layer 1:** Dense layer with 512 units and ReLU activation.
 - Followed by **Batch Normalization** to stabilize and speed up training.
 - Followed by **Dropout (0.2)** to reduce overfitting.
- **Hidden Layer 2:** Dense layer with 256 units and ReLU activation.
- **Hidden Layer 3:** Dense layer with 128 units and ReLU activation.
- **Output Layer:** Dense layer with 10 units and softmax activation, giving class probabilities for digits 0–9.

Training Configuration

The model is compiled and trained as follows:

```
model.compile(
optimizer=optimizers.Adam(learning_rate=0.001),
loss='sparse_categorical_crossentropy',
metrics=['accuracy']
)
```

```
history = model.fit(
x_train_flat, y_train,
epochs=20,
batch_size=128,
validation_split=0.1,
verbose=1
)
```

Key points:

- **Loss function:** `sparse_categorical_crossentropy` (labels are integer-encoded).
- **Optimizer:** Adam with learning rate 0.001, a widely used adaptive optimizer for deep learning.[5]
- **Metrics:** Accuracy is used as the primary performance metric.
- **Training protocol:**
 - 20 epochs

- Batch size of 128
- 10% of the training data held out as a validation set (`validation_split=0.1`).

Test Evaluation

After training, the model is evaluated on the test set:

```
test_loss, test_acc = model.evaluate(x_test_flat, y_test, verbose=0)
```

The code prints:

```
final
baseline (Logistic Regression) Accuracy: XX.XX%
MLP test Accuracy: YYYY%
```

Where:

- XX.XX% is the baseline Logistic Regression accuracy.
- YYYY% is the MLP test accuracy.

In typical experiments on MNIST, similar MLP architectures achieve around **97–98%** test accuracy, significantly outperforming the linear baseline.[1][5]

Confusion Matrix Analysis

To analyze classification errors in more detail, a confusion matrix is computed using the MLP predictions:

```
y_pred = np.argmax(model.predict(x_test_flat), axis=1)
cm = confusion_matrix(y_test, y_pred)
```

The confusion matrix is visualized with Seaborn as a heatmap:

```
plt.figure(figsize=(10,8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title('MLP Confusion Matrix: Handwritten Digit Recognition')
plt.xlabel('Predicted Digit')
plt.ylabel('Actual Digit')
plt.show()
```

Interpretation

- The diagonal elements represent the number of correctly classified examples for each digit class (0–9).
- Off-diagonal entries represent misclassifications, indicating which digits the model confuses with each other.

Typical patterns on MNIST include:

- Confusion between **3 and 5**, **4 and 9**, or **7 and 9**, due to similar writing styles.[1][6]
- Very high diagonal values, reflecting the strong overall performance of the MLP.

By examining the confusion matrix, we can identify specific digits that might benefit from:

- Additional training data,

- Data augmentation, or
- Architectural changes that capture particular shape variations more effectively.

Training and Validation Loss Curves

During training, the loss history is stored in `history.history`. The following code plots training and validation loss over 20 epochs:

```
plt.figure(figsize=(8,5))
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('MLP Loss Metrics over 20 Epochs')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.show()
```

Observations

From such curves, we typically look for:

- **Convergence:** Training loss decreasing steadily over epochs, indicating that the model is learning.
- **Generalization:** Validation loss following training loss and not increasing sharply, indicating limited overfitting.
- **Overfitting detection:** If validation loss starts increasing while training loss continues to decrease, it suggests overfitting; techniques such as stronger regularization, earlier stopping, or more dropout may be helpful.[5][7]

In this project, the presence of batch normalization and dropout is intended to help maintain good generalization and stable training behavior.

Comparison of Logistic Regression and MLP

Accuracy Comparison

The project directly compares test accuracies:

- **Logistic Regression baseline:** `baseline_acc * 100%`
- **MLP model:** `test_acc * 100%`

Empirically, the MLP achieves noticeably higher accuracy than the baseline. For example, if the baseline achieves around 90–92%, the MLP often reaches around 97–98% on MNIST, representing a significant performance gain.[3][5]

Model Capacity and Non-Linearity

The key reasons for the MLP's superior performance include:

1. Non-linear feature learning:

ReLU activations in multiple hidden layers allow the network to learn complex non-linear decision boundaries that better separate digit classes in the high-dimensional pixel space.

2. **Representation learning:**

The hidden layers automatically learn intermediate representations (e.g., strokes, edges, digit shapes) directly from raw pixels, whereas Logistic Regression uses only a single linear transformation of the input.

3. **Regularization:**

Batch normalization and dropout help control overfitting and improve generalization, which is essential when using a high-capacity model.

In contrast, Logistic Regression is limited to linear decision boundaries in the original feature space, which is not expressive enough to fully separate all digit classes with high accuracy.

Conclusions

This project demonstrates a complete workflow for handwritten digit recognition on the MNIST dataset, from data loading and preprocessing to model training, evaluation, and error analysis.

Key conclusions:

1. **Baseline performance:**

The multinomial Logistic Regression model provides a strong linear baseline but is limited in capturing complex digit variations.

2. **Deep learning improvement:**

The Multilayer Perceptron significantly outperforms the baseline on MNIST, achieving higher test accuracy thanks to its deeper architecture and non-linear activations.

3. **Regularization and stability:**

Batch normalization and dropout improve training stability and generalization, as reflected in the training/validation loss curves.

4. **Error analysis:**

The confusion matrix reveals specific digit pairs that are frequently confused, offering insights for future improvements (e.g., data augmentation or architectural changes).

Future Work

Possible extensions of this project include:

- Replacing the MLP with a **Convolutional Neural Network (CNN)** that exploits the spatial structure of images and typically achieves above 99% accuracy on MNIST.[1][5]
- Performing **hyperparameter tuning** (number of layers, layer sizes, learning rate, dropout rate) to further improve performance.
- Applying **data augmentation** (rotations, shifts, scaling) to increase robustness to handwriting variations.
- Comparing additional models such as Support Vector Machines or more advanced deep architectures.

References

- [1] LeCun, Y., Cortes, C., & Burges, C. J. C. (2010). MNIST handwritten digit database. <http://yann.lecun.com/exdb/mnist/>
- [2] BotPenguin. (2025). *What is MNIST Dataset and How does it Work?* <https://botpenguin.com/glossary/mnist-dataset>
- [3] Reddit. (2019). *I trained a logistic regression model on the MNIST dataset and it got ...* <https://www.reddit.com/r/MLQuestions/comments/d4xe4v>
- [4] Kaggle. (n.d.). *Understanding Baseline Models*. <https://www.kaggle.com/getting-started/314021>
- [5] DigitalOcean. (2022). *MNIST Dataset in Python - Basic Importing and Plotting*. <https://www.digitalocean.com/community/tutorials/mnist-dataset-in-python>
- [6] Boehmke, B. (2020). *Hands-On Machine Learning with R* (Chapter 5: Logistic Regression). <https://bradleyboehmke.github.io/HOML/logistic-regression.html>
- [7] Packt Publishing. (2024). *Multilayer Perceptron (MLP)*. <https://subscription.packtpub.com/book/programming/9781838821654/1/ch01lv1sec04/3-multilayer-perceptron-mlp>